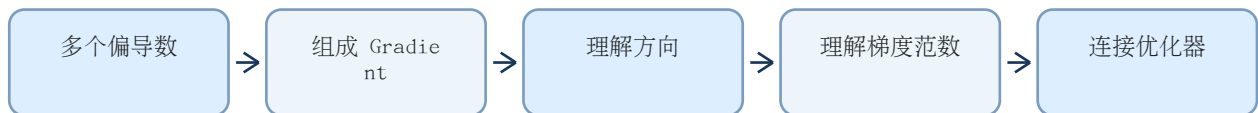


第20讲：Gradient（梯度）

怎样把所有偏导数组成一个方向？

核心概念：【必须真正掌握】

本讲只要求建立 Gradient 的必要直觉：它不是一个神秘的新公式，而是把第19讲得到的很多偏导数组织成与参数结构对应的一组数，并用来描述 Loss 在当前点附近最敏感的方向。



建议学习顺序：先看讲义 -> 手算二维例子 -> 运行方向实验 -> 做练习 -> 用速查卡复盘。

一、本讲学习目标

把第19讲的多个偏导数组织成 Gradient，并理解 Gradient 与参数 Shape 的关系。

能够读懂 `L`、`gradient`、`grad`、`gradient norm` 等论文和代码术语。

理解梯度在欧氏距离意义下指向当前点局部上升最快方向，负梯度指向局部下降最快方向。

理解 Gradient 只提供局部方向信息，不等价于“直接告诉你全局最优参数在哪里”。

理解学习率/步长与梯度是两个不同概念：方向正确，步长过大也可能让 Loss 上升。

理解标量、向量、矩阵参数的 Gradient 通常与对应参数具有相同 Shape。

把 Gradient 重新连接到 LLM 训练：Forward 得到 Loss，Backward 计算参数梯度，Optimizer 再决定如何更新参数。

核心概念：与第19讲的关系

第19讲一次只看一个偏导数；第20讲不再把这些偏导数散落着看，而是把它们按参数结构收集起来。这一步就是从 Partial Derivative 走向 Gradient。

二、【原理】Gradient 到底是什么？

假设 Loss 同时依赖三个参数：

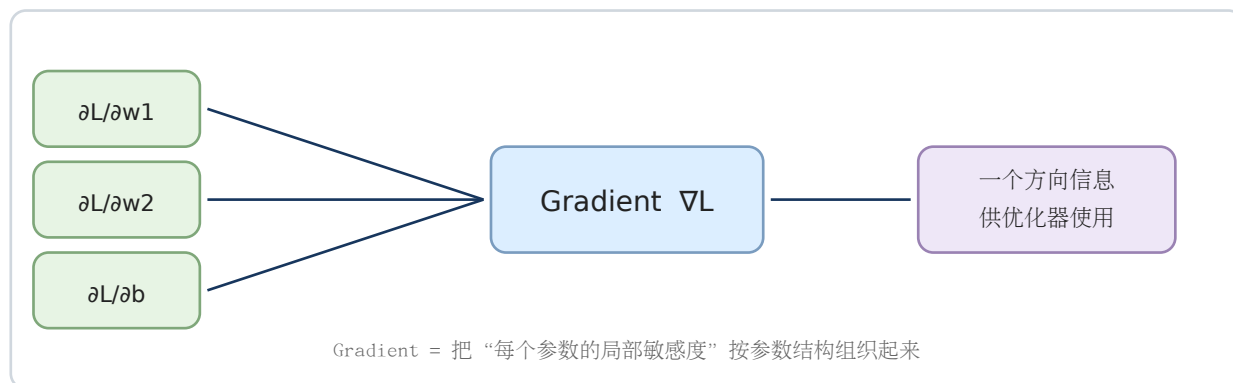
$$L = L(w1, w2, b)$$

第19讲分别得到：

$$\partial L / \partial w1 \quad \partial L / \partial w2 \quad \partial L / \partial b$$

Gradient 做的事情很朴素：把这些偏导数按顺序放在一起。

$$\nabla L = [\partial L / \partial w1, \partial L / \partial w2, \partial L / \partial b]$$



数学直觉：不要把 当成神秘运算

你可以先把 ∇L 读成：“Loss 对当前所有参数的偏导数集合”。在简单向量参数情况下，它就是一个向量；在真实神经网络框架里，每个参数 Tensor 会有一个同 Shape 的 gradient Tensor。

三、【具体数字例子】从两个偏导数组成一个梯度

使用一个故意设计得很简单的二维 Loss:

$$L(w1, w2) = (w1 - 2)^2 + 2(w2 + 1)^2$$

选当前参数:

$$w = [0, 1]$$

当前 Loss:

$$L = (0-2)^2 + 2(1+1)^2 = 4 + 8 = 12$$

分别求两个偏导:

$$\partial L / \partial w1 = 2(w1 - 2) = -4$$

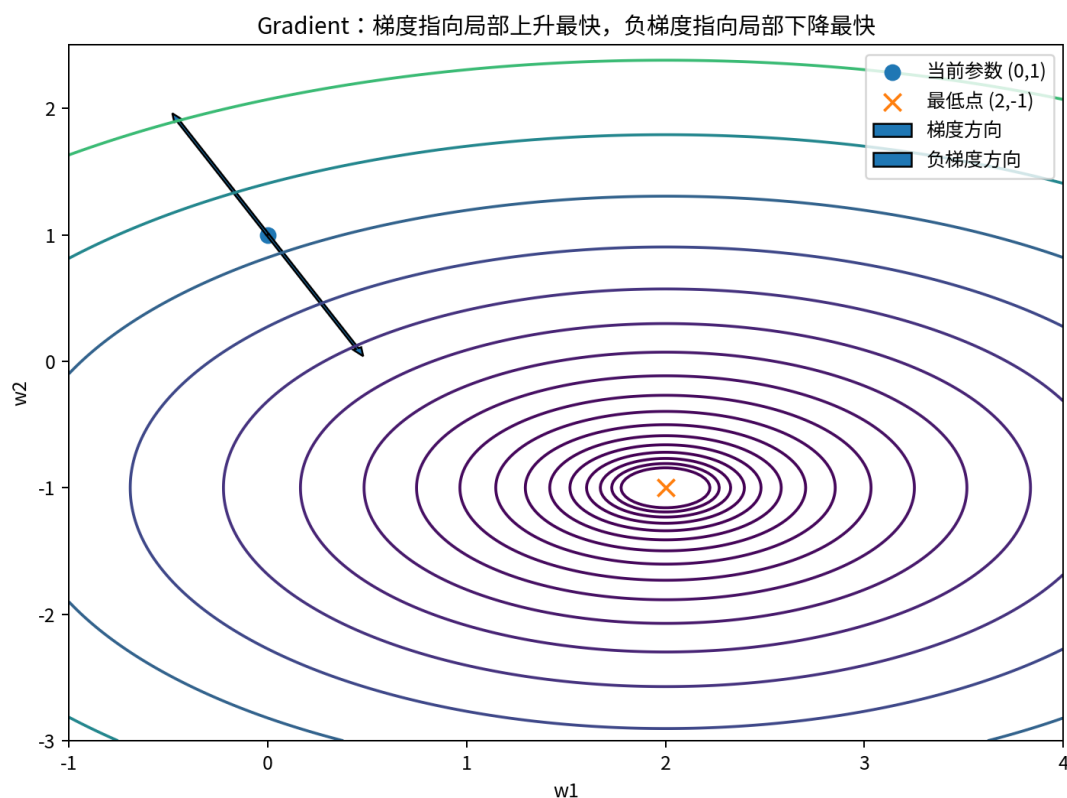
$$\partial L / \partial w2 = 4(w2 + 1) = 8$$

因此:

$$\nabla L = [-4, 8]$$

量	当前值	自然语言解释
L/ w1	-4	只增加一点 w1, Loss 倾向下降; 只减小一点 w1, Loss 倾向上升。
L/ w2	8	只增加一点 w2, Loss 倾向上升; 只减小一点 w2, Loss 倾向下降。
L	[-4, 8]	把两个坐标轴上的局部敏感度组合成一个整体方向。

四、【图解】梯度为什么能表示“方向”？



把二维参数空间想成一张地形图：横轴是 w_1 ，纵轴是 w_2 ，等高线代表相同 Loss。当前点的梯度箭头指向 Loss 局部增长最快的方向；反方向 $-L$ 则指向局部下降最快方向。

数学直觉：“最快”有一个前提

这里的“最快”说的是：在当前位置附近、允许你移动相同的很小欧氏距离时，哪个方向让 Loss 增加最多。它是局部的一阶结论，不是在承诺一步就到全局最低点。

五、【为什么是最快方向】用方向实验而不是先做证明

本课程不要求先学方向导数和完整多元微积分证明。我们先用实验建立直觉：从同一个点出发，选 721 个不同方向，每个方向都只走非常小的一步，然后测量 Loss 的变化率。

当前梯度：

$$\nabla L = [-4, 8]$$

归一化以后：

$$\nabla L / \|\nabla L\| \approx [-0.4472, 0.8944]$$

程序搜索到的最快上升方向约为：

$$[-0.4462, 0.8949]$$

两者夹角余弦约为：

$$0.999999$$

越接近 1，说明两个方向越接近。负梯度方向的实验结果同样约为 0.999999。

具体例子：先记直觉

Gradient 的每个分量来自一个偏导；这些分量一起决定了“在当前点附近，朝哪个组合方向改变所有参数，Loss 增长最敏感”。

六、【必要公式】Gradient Norm 是什么？

如果梯度是二维向量：

$$\nabla L = [g1, g2]$$

它的长度（norm）通常写成：

$$||\nabla L|| = \text{sqrt}(g1^2 + g2^2)$$

本例：

$$||\nabla L|| = \text{sqrt}((-4)^2 + 8^2) = \text{sqrt}(80) \approx 8.9443$$

直觉上，梯度方向告诉“往哪里最敏感”，梯度范数告诉“当前局部有多陡”。

概念	回答的问题	注意
Gradient direction	Loss 在当前点附近最敏感的组合方向是什么？	是局部方向，不是全局答案。
Gradient norm	这个局部坡度整体有多大？	会受参数尺度、Loss 定义等影响。
Gradient = 0	一阶局部坡度为 0	可能是极小值、极大值、鞍点或平坦区域。

常见误区 / 风险：常见误区

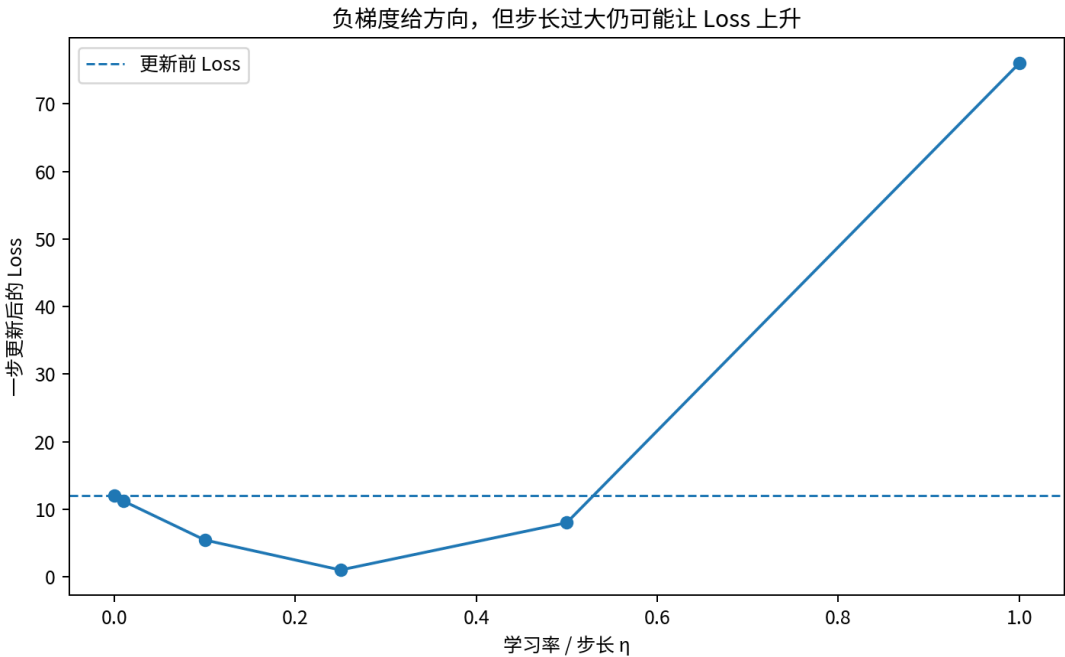
“Gradient norm 很大，所以这个参数/特征在现实世界很重要” 并不成立。梯度是模型当前参数化、当前样本/Batch、当前 Loss 和数值尺度下的局部量。

七、【Gradient 与更新不是一回事】为什么还需要 Learning Rate?

如果想让 Loss 下降，一个最直接的想法是沿负梯度方向移动：

$$w_new = w - \eta \nabla L$$

其中 η (eta) 就是步长/learning rate。梯度提供方向和局部斜率， η 决定你一次走多远。



η	一步后的 Loss	结论
0.01	11.2144	下降，但走得很小。
0.10	5.4400	明显下降。
0.25	1.0000	这个例子里下降很多。
0.50	8.0000	仍下降，但已经跨过一个方向的最低位置。
1.00	76.0000	步长过大，Loss 反而暴涨。

核心概念：先不要把本讲变成优化器课

本讲只要求看懂：负梯度是局部下降方向，但“往这个方向走多远”是另一个问题。第33讲以后会系统学习 Gradient Descent、SGD、Momentum、Adam/AdamW。

八、【代码实验】NumPy 如何表示 Gradient?

最直接的代码就是先计算各偏导，再放进数组：

```
w = np.array([0.0, 1.0])

dw1 = 2.0 * (w[0] - 2.0)
dw2 = 4.0 * (w[1] + 1.0)
grad = np.array([dw1, dw2])

print(grad)          # [-4.  8.]
print(grad.shape)    # (2,)
```

也可以完全不知道解析公式，只用数值差分逐个参数估计：

```
def numerical_gradient(func, w, eps=1e-5):
    grad = np.zeros_like(w, dtype=float)
    for i in range(w.size):
        w_plus = w.copy()
        w_minus = w.copy()
        w_plus[i] += eps
        w_minus[i] -= eps
        grad[i] = (func(w_plus) - func(w_minus)) / (2 * eps)
    return grad
```

本讲实验中解析梯度与数值梯度最大误差约为 $1e-10$ 。

科研连接：为什么不能给十亿参数都这样数值差分？

因为每个参数都要额外做多次 Forward。真实 LLM 参数量巨大，这会贵得不可接受。后面的 Backpropagation/Autograd 会复用计算图，以高得多的效率得到所有需要的梯度。

九、【参数是什么 Shape，Gradient 通常就是什么 Shape】

前面的参数 w 是一个长度为 2 的向量，所以它的梯度也是长度为 2:

$$w.shape = (2,) \rightarrow grad_w.shape = (2,)$$

如果参数是一个矩阵:

$$W.shape = (Din, Dout)$$

那么对这个矩阵里每一个元素 w_{ij} 都有一个偏导:

$$\partial L / \partial w_{ij}$$

把这些偏导放回原来的位置，就得到一个与 W 同 Shape 的梯度矩阵:

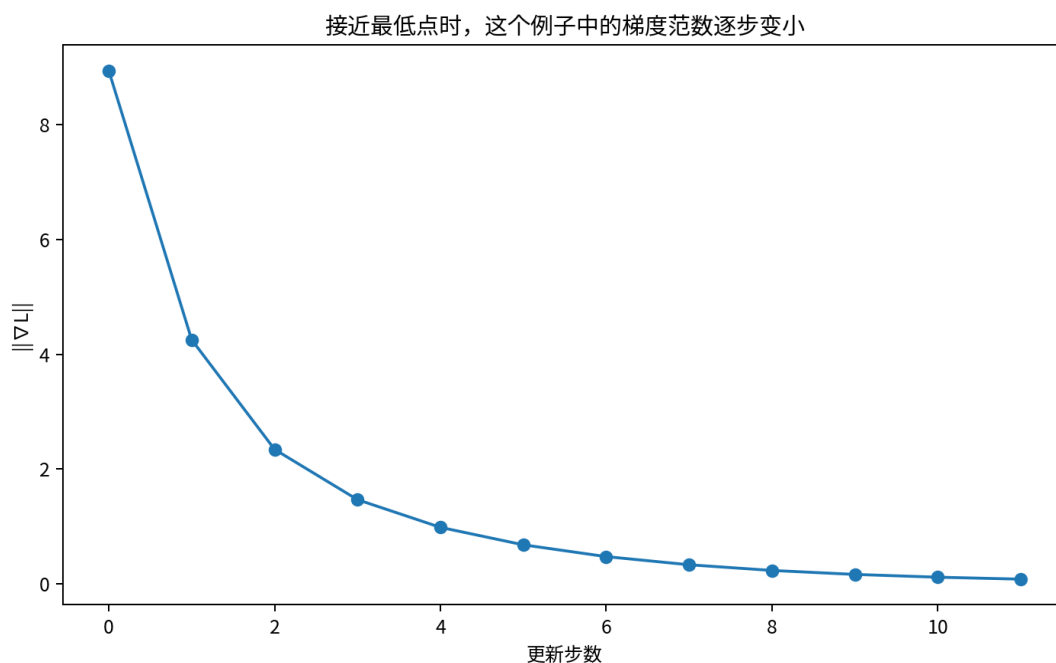
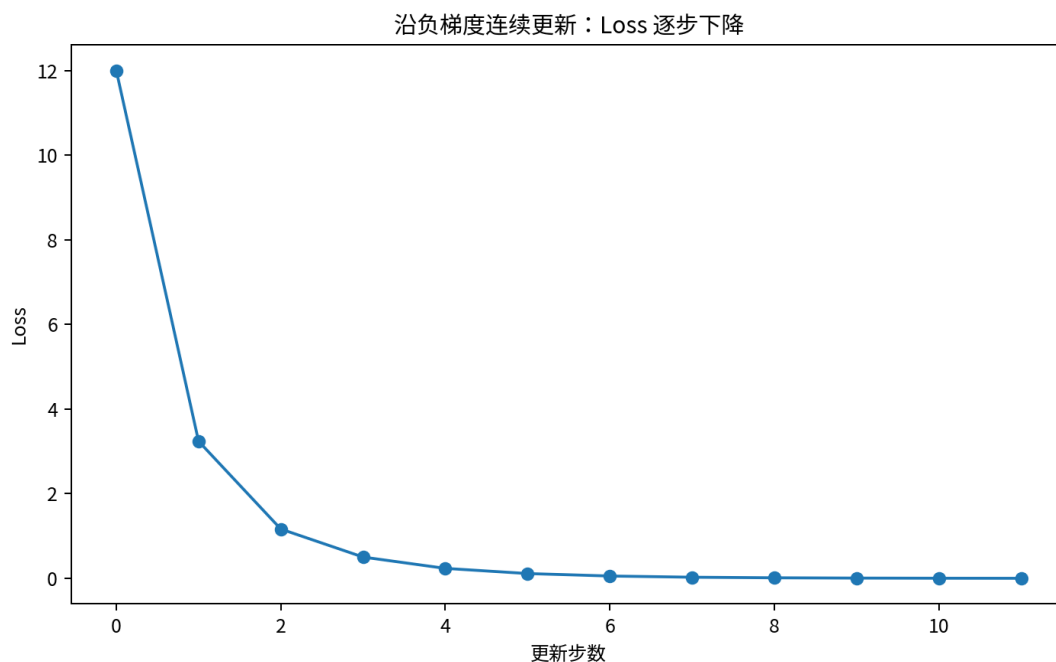
$$grad_W.shape = W.shape$$

本讲实验5:

Tensor	Shape	含义
X	(3, 2)	3个样本，每个2个输入特征
W	(2, 2)	Linear 的4个可训练权重
Prediction	(3, 2)	每个样本2个输出
grad_W	(2, 2)	4个权重各自对应的 $\partial L / \partial w_{ij}$

这条 Shape 规律以后非常实用：一个参数 Tensor 的 ``.grad``，通常要能与这个参数逐元素对齐，否则优化器不知道应该更新哪个元素。

十、【连续更新实验】为什么越接近低谷，梯度可能越小？



在本讲这个简单的凸二次函数中，用 $\eta=0.15$ 连续沿负梯度更新，Loss 从 12.0 降到约 0.00156，梯度范数也从 8.944 降到约 0.079。

常见误区 / 风险：不要过度推广

“越接近好解，梯度一定越小” 不是所有深度网络训练轨迹都能简单这样描述。真实 Loss landscape 可能有鞍点、噪声、不同尺度、mini-batch 波动和复杂曲率。本实验只是建立最基本的一阶直觉。

十一、【与 LLM / Transformer 的关系】十亿参数的 Gradient 是什么样？

Transformer 中的参数并不是一个大向量真的手工拼在一起，而是分散在许多 Tensor 中，例如：

参数	典型 Shape（示意）	它的 Gradient
W_Q	(D, D)	同 Shape 的 L/ W_Q
W_K	(D, D)	同 Shape 的 L/ W_K
W_V	(D, D)	同 Shape 的 L/ W_V
FFN W1	(D, D_ff)	同 Shape 的 L/ W1
Embedding table	(V, D)	同 Shape 的 embedding gradient

概念上，你仍然可以把“所有可训练参数的所有偏导数”看成一个巨大的 Gradient。训练时：



这和你以前 LLM 课程里见过的 Pretraining 已经接起来了：Next-token Loss 不是终点，它还要通过 Gradient 把“错了多少”转换成“哪些参数应该往什么局部方向调整”。

科研连接：Agent Systems 连接

做 Agent Systems 实验时，你多数时候不会从零训练底座 LLM，但你仍会遇到微调、LoRA/QLoRA、reward model、policy optimization 或小模型训练。能读懂 gradient、grad norm、gradient clipping 等日志，是判断训练是否正常的最低模型基础。

十二、【常见误区】

常见误区 / 风险：误区1: Gradient 就是参数更新量

不对。Gradient 是局部敏感度/方向信息；真正更新量还取决于 learning rate、Momentum、Adam 的状态、weight decay 等。

常见误区 / 风险：误区2: 沿负梯度走一步一定让 Loss 下降

只有足够小的局部步长才有这种一阶直觉。步长过大可以直接跨过低谷。

常见误区 / 风险：误区3: Gradient 指向最低点

不对。它指向当前点局部上升最快方向；负梯度只是当前点局部下降最快方向。

常见误区 / 风险：误区4: Gradient=0 就说明训练成功

不一定。它可能出现在局部极小、局部极大、鞍点或平坦区域，还要结合 Loss、任务指标和训练状态判断。

常见误区 / 风险：误区5: Gradient 很大说明特征重要

不能这样做因果或全局解释。梯度依赖参数化、尺度、当前数据和当前模型状态。

常见误区 / 风险：误区6: 每个神经网络只有一个 Gradient 数字

不对。每个可训练参数元素都有对应偏导；框架通常按参数 Tensor 保存同 Shape gradient。

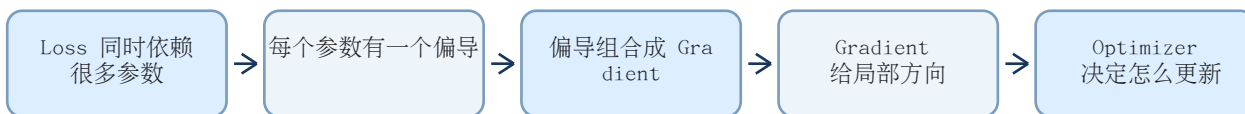
十三、【公式逐项拆解】把 L 翻译成自然语言

$$\nabla_w L = [\partial L/\partial w_1, \partial L/\partial w_2, ..., \partial L/\partial w_n]$$

符号	你应该怎么读	不要误解成
L	当前定义的 Loss / Objective	现实世界的“真实价值”
w	所有被关注的参数	只有一个标量 weight
L/w_i	其他变量暂时固定时， w_i 微变对 Loss 的一阶局部影响	w_i 的因果重要性
wL	把所有这些局部敏感度组织起来	直接的最终更新量
$-wL$	局部最快下降方向	全局最短路线

以后读论文看到 “we compute gradients with respect to model parameters”，最朴素的翻译就是：对每个可训练参数计算 Loss 的偏导，并把这些信息交给优化过程。

十四、本讲知识闭环



现在把第13-20讲连起来:

Prediction -> Loss -> Derivative -> Partial Derivative -> Gradient

下一步要回答的问题是:

核心概念: 新的关键问题

真实神经网络不是 $L(w_1, w_2)$ 这种一步公式, 而是 $x \rightarrow \text{Linear} \rightarrow \text{ReLU} \rightarrow \text{Linear} \rightarrow \text{Loss}$ 的长计算链。Loss 对前面某个参数的影响, 要怎样穿过中间节点一路传回来?

这就需要后面几讲的 Chain Rule、Computational Graph 和 Backpropagation。